

?? Sistema de Análise de Luz Noturna via Satélite

Documentação Completa das Funcionalidades

****Data:**** Maio de 2026

****Versão:**** 2.0 Final

****Tipo:**** Aplicação Web + CLI para Processamento de Imagens de Satélite

? Índice

1. [Visão Geral](#visão-geral)

2. [Arquitetura do Sistema](#arquitetura-do-sistema)

3. [Funcionalidades Principais](#funcionalidades-principais)

4. [Componentes Técnicos](#componentes-técnicos)

5. [APIs REST](#apis-rest)

6. [Interface Web](#interface-web)

7. [Processamento de Dados](#processamento-de-dados)

8. [Análise e Relatórios](#análise-e-relatórios)

9. [Automação](#automação)

10. [Guias de Uso](#guias-de-uso)

Visão Geral

Este é um sistema completo de **análise de iluminação noturna urbana** utilizando imagens de

satélite TIFF (GeoTIFF). O sistema processa centenas de imagens de múltiplas cidades da região do Vale do Itajaí em Santa Catarina, detectando e analisando crescimento de luz ao longo de 10 anos (2014-2024).

Características Principais

- ? **Análise de 1200+ imagens de satélite**
- ? **Processamento paralelo de 8-10x mais rápido**
- ? **Cobertura de 60+ cidades**
- ? **Histórico de 10 anos (2014-2024)**
- ? **Interface web moderna e responsiva**
- ? **Detecção automática de crescimento de luz**
- ? **Geração de gráficos e tendências**
- ? **API REST completa**
- ? **Sistema de cache inteligente**

- ? ****Automação por agendador de tarefas****
- ? ****Sugestão automática de thresholds****
- ? ****Comparação periódica de imagens****

Arquitetura do Sistema

...

??

? Interface Web (Flask) ?

? ?? app.py (Aplicação principal) ?

? ?? templates/index.html (Interface responsiva) ?

? ?? static/ (CSS, JavaScript, assets) ?

??

?

??

? Camada de Processamento ?

? ?? leitura_de_luz.py (Análise de intensidade) ?

? ?? processador_paralelo.py (Paralelo com threads) ?

? ?? tendencias.py (Análise de tendências) ?

? ?? sugestores_threshold.py (Sugestão de limiares) ?

? ?? comparador_crescimento.py (Comparação de crescimento) ?

??

?

??

? Armazenamento de Dados ?

? ?? resultados_*.csv (Dados processados) ?

? ?? processados_*.json (Cache de imagens) ?

? ?? coordenadas_*.json (Coordenadas de cidades) ?

??

...

Funcionalidades Principais

1. ? Interface Web Interativa

Uma aplicação Flask moderna com Bootstrap 5 e Leaflet.js para visualização geográfica.

****Recursos:****

- ?? **Mapa interativo** das cidades com marcadores
- ? **Seletor de cidades/recortes** por dropdown
- ? **Visualização de imagens** organizadas por ano
- ? **Análise individual de TIFF**
- ? **Comparação entre períodos**
- ? **Upload de arquivos personalizados**
- ? **Painel de sugestão de thresholds**
- ? **Recomendações automáticas**

URL: `http://localhost:5000`

2. ? Processamento de Imagens TIFF

Análise completa de imagens geoespaciais com detecção de anomalias.

Validações Implementadas

****Detecção de Pixels Nulos:****

- Identifica pixels com valor 0 (erro do sensor)
- Detecta NaN (valores indefinidos)
- Diferencia entre erros reais e NoData legítimo (-9999)

****Detecção de Outliers:****

- Threshold fixo de 250 para intensidade luminosa
- Identifica anomalias do sensor
- Relatório detalhado de qualidade

****Cálculo de Estatísticas:****

- Média aritmética de intensidade
- Mediana (valor central)
- Desvio padrão (variação dos dados)
- Percentis (25%, 75%, etc)
- Intervalo Interquartil (IQR)

Métricas Retornadas

```
```json
```

```
{
```

```
 "total_pixels": 250000,
```

```
 "pixels_validos": 248750,
```

"percentual\_valido": "99.50%",

"intensidade\_media": 45.32,

"intensidade\_mediana": 42.15,

"intensidade\_minima": 1.20,

"intensidade\_maxima": 248.95,

"desvio\_padrao": 28.45,

"status": "? ACEITA"

}

...

---

### 3. ? Processamento Paralelo

Sistema de processamento multi-thread para velocidade 8-10x maior.

**\*\*Características:\*\***

- Utiliza ThreadPoolExecutor com N workers
- Processa múltiplas imagens simultaneamente
- Sistema de cache inteligente (processados.json)
- Evita reproprocessamento desnecessário
- Progresso em tempo real

**\*\*Tempo de Processamento:\*\***

- Sequencial: ~40 minutos (120 imagens)
- Paralelo: ~5-8 minutos (120 imagens)
- **\*\*Ganho: 500-800% mais rápido\*\***

---

### 4. ? Análise de Tendências

Geração automática de gráficos e análises estatísticas.

**\*\*Tipos de Análise:\*\***

#### Tendência de 10 Anos

...

Gráfico mostrando:

- Evolução da intensidade média por ano
- Linha de tendência (regressão linear)
- Intervalo de confiança

- Percentual de crescimento total

...

#### Comparação Primeiro vs Último

...

Gráfico lado a lado:

- Primeira imagem do período (2014)

- Última imagem do período (2024)

- Diferença de intensidade

- Percentual de mudança

...

#### Distribuição Estatística

...

- Histograma de intensidades

- Caixa de distribuição (boxplot)

- Curva de densidade

...

---

### ### 5. ? Sugestão de Thresholds

Análise inteligente para sugerir valores de limiar para detecção de crescimento de luz.

**\*\*4 Níveis de Sugestão:\*\***

1. **\*\*Conservador\*\*** - ? Thresholds muito cautelosos

- Detecta apenas crescimentos claros
- Recomendado para análises preliminares

2. **Moderado** - ?? Equilíbrio entre sensibilidade e especificidade

- Recomendado para análises padrão
- Maioria dos casos de uso

3. **Agressivo** - ? Alta sensibilidade

- Detecta mudanças pequenas também
- Recomendado para estudos detalhados

4. **Alto Crescimento** - ? Apenas crescimentos significativos

- Máxima sensibilidade
- Detecção de anomalias extremas

**\*\*Cada sugestão inclui:\*\***

- Valor do threshold em intensidade de luz
- Percentil estatístico correspondente
- Justificativa técnica
- Recomendação baseada em contexto

---

### ### 6. ? Comparação de Crescimento

Análise comparativa entre duas imagens.

**\*\*Métricas Calculadas:\*\***



- Diferença de intensidade bruta
- Percentual de crescimento relativo
- Normalização por variância
- Status: CRESCIMENTO/DECRÉSCIMO/ESTÁVEL
- Significância estatística

**\*\*Exemplo:\*\***

'''

Comparação: 2014 vs 2024

?? Intensidade 2014: 32.45

?? Intensidade 2024: 47.82

?? Crescimento: +15.37 (+47.3%)

?? Status: CRESCIMENTO SIGNIFICATIVO

?? Threshold: Excede 2.3 desvios padrão

'''

---

### 7. ? Gerenciamento de Cidades

Suporte para 60+ cidades com descoberta automática.

**\*\*Cidades Cobertas:\*\***

**\*Costeiras:\***

- Balneário Camboriú, Itajaí, Itapema, Navegantes
- Penha, Porto Belo, Bombinhas, Barra Velha

**\*Vale Central:\***

- Brusque, Gaspar, Blumenau, Pomerode

- Botuverá, Guabiruba, Luiz Alves

**\*Alto Vale:\***

- Indaial, Timbó, Ibirama, Apiúna

- Rodeio, Rio dos Cedros, Blumenau

**\*Sudoeste/Sul:\***

- Rio do Sul, Agrolândia, Atlanta

- Petrolândia, Aurora, Ituporanga

**\*\*Coordenadas Precisas:\*\***

- Obtidas de IBGE (centroide oficial)

- Verificadas com Nominatim/OpenStreetMap

- Resolução de ~50m

---

### 8. ? Menu Principal (CLI)

Interface de linha de comando com 5 opções principais.

'''

MENU PRINCIPAL

=====

1 - Processar todas as imagens

2 - Ver tendências e gráficos

3 - Configurar automação

4 - Ver status de processamento

5 - Limpar cache

...

---

## Componentes Técnicos

### leitura\_de\_luz.py

**\*\*Classe: AnalisadorLuzSatelite\*\***

Principal mecanismo de análise de imagens.

```
```python
```

```
analizador = AnalisadorLuzSatelite(
```

```
    metodo_outlier='iqr',    # 'iqr' ou 'zscore'
```

```
    limiar_iqr=3.0,          # Sensibilidade IQR
```

```
    limiar_zscore=3.0,       # Sensibilidade Z-score
```

```
    modo_geoespacial=True    # Diferencia NoData
```

```
)
```

```
# Processar imagem
```

```
stats = analizador.processar_imagem(imagem_array)
```

```
# Comparar crescimento
```

```
comparacao = analizador.comparar_crescimento_luz(stats_ant, stats_atu)
```

'''

****Métodos Principais:****

- `processar\_imagem()` - Análise completa de uma imagem
- `comparar\_crescimento\_luz()` - Comparação entre imagens
- `gerar\_relatorio()` - Formatação de relatório
- `validar\_pixels()` - Validação e limpeza de dados
- `detectar\_outliers()` - Identificação de anomalias

processador_paralelo.py

****Classe: ProcessadorParalelo****

Orquestração de processamento multi-thread.

```
```python
```

```
processador = ProcessadorParalelo()
```

```
processador.processar_pasta(caminho)
```

```
```
```

****Otimizações:****

- ThreadPoolExecutor com limite de workers

- Fila de tarefas gerenciada

- Progresso em tempo real

- Tratamento automático de erros

- Retry para imagens problemáticas

tendencias.py

****Classe: AnalisadorTendencias****

Análise estatística e geração de gráficos.

```python

analizador = AnalisadorTendencias()

relatorio = analizador.gerar\_relatorio(arquivo\_csv)

```

****Relatório Inclui:****

- Dados gerais (período, cidades)
- Análise por ano
- Tendência geral (regressão linear)
- Gráficos PNG
- Recomendações

sugestores_threshold.py

****Classe: SugestorThreshold****

Sistema de sugestão inteligente de limiar.

```
```python
```

```
sugestor = SugestorThreshold()
```

```
sugestor.ler_csv('resultados_cidade.csv')
```

```
sugestor.calcular_stats_anuais()
```

```
sugestoes = sugestor.sugerir_thresholds()
```

```
```
```

```
**Retorna:**
```

```
```python
```

```
{
```

```
 'periodos': {'2014-2018': {...}, '2019-2024': {...}},
```

```
 'crescimento': {...},
```

```
 'thresholds_sugeridos': {
```

'conservador': {...},

'moderado': {...},

'agressivo': {...},

'alto\_crescimento': {...}

},

'estatisticas\_gerais': {...},

'recomendacoes': [...]

}

...

---

### app.py

## **\*\*Aplicação Flask Principal\*\***

Servidor web com API REST completa.

### **\*\*Configurações:\*\***

- HOST: localhost

- PORT: 5000

- MAX\_UPLOAD: 50MB

- AUTO-DISCOVERY: Pastas de cidades

---

**## APIs REST**

### ### Endpoints Disponíveis

#### #### 1. GET /

Página principal (HTML)

---

#### #### 2. GET /api/listar-imagens

Lista todas as imagens disponíveis

**\*\*Resposta:\*\***

``json

[

{

"ano": "2014",

"imagens": [

{

"nome": "imagem\_01.tif",

"caminho": "/path/to/file",

"tamanho\_kb": 1024.5

}

],

"total": 12

}

]

...

---

### #### 3. POST /api/analisar

Analisa arquivo TIFF selecionado

**\*\*Request:\*\***

```json

{

"caminho": "/path/to/file.tif"

}

```

**\*\*Resposta:\*\***

```json

{

"sucesso": true,

"nome_arquivo": "imagem.tif",

"dimensoes": "500 × 500",

"total_pixels": "250,000",

"pixels_validos": "248,750",

"percentual_valido": "99.50%",

"intensidade_media": "45.32",

"intensidade_mediana": "42.15",

"status": "? ACEITA",

"status_class": "success"

}

...

4. POST /api/analisar-upload

Analisa arquivo TIFF enviado

****Request (multipart/form-data):****

- file: arquivo .tif/.tiff

****Resposta:**** Mesma da análise de arquivo

5. POST /api/comparar-crescimento

Compara crescimento entre dois arquivos

****Request:****

```json

{

"arquivo\_anterior": "/path/1.tif",

"arquivo\_atual": "/path/2.tif"

}

```

****Resposta:****

```json

{

"sucesso": true,

"crescimento\_media": 15.37,

"percentual\_crescimento": 47.3,

"status\_crescimento": "CRESCIMENTO",

"crescimento\_significativo": true,

"detalhes": "..."

}

...

---

#### #### 6. GET /api/sugerir-thresholds

Sugestão de thresholds para cidade atual

**\*\*Resposta:\*\***

```json

{

"sucesso": true,

"cidade": "Blumenau",

"dados": {

"periodos": {...},

"crescimento": {...},

"thresholds_sugeridos": {...},

"recomendacoes": [...]

}

}

...

7. GET /api/pastas-cidades

Lista todas as cidades/recortes disponíveis

****Resposta:****

```json

[

{

"id": "Blumenau\_recorte",

"nome\_amigavel": "Blumenau - Recorte",

"cidade": "Blumenau",

"total\_imagens": 145,

"coordenadas": [-26.9196, -49.0658]

}

]

```

8. POST /api/selecionar-pasta

Seleciona a pasta de trabalho atual

****Request:****

```json

{

"pasta": "Blumenau\_recorte"

}

```

Interface Web

Componentes Principais

1. Mapa Interativo

- **Biblioteca:** Leaflet.js 1.9.4
- **Provider:** OpenStreetMap
- **Zoom Padrão:** 11
- **Centro:** Vale do Itajaí (-27.079, -49.063)
- **Funcionalidade:** Clique em marcador = seleção de cidade

2. Seletor de Cidades

- Dropdown com todas as cidades descobertas automaticamente

- Mostra nome amigável e total de imagens
- Atualiza mapa ao selecionar

3. Listagem de Imagens

- Organizadas por ano (2014-2024)
- Expandir/Recolher por ano
- Mostra tamanho em KB
- Botão de análise por imagem

4. Painel de Análise Individual

- Exibição de todas as métricas
- Status com ícone colorido (? ACEITA, ? VERIFICAR, ? REJEITADA)
- Gráfico visual de qualidade
- Botão para comparar com outra imagem

5. Painel de Comparação

- Seletor de duas imagens
- Gráfico de comparação lado a lado
- Análise de crescimento
- Seta indicando crescimento (? acima, ? abaixo, ? estável)

6. Painel de Sugestão de Thresholds

- ? Botão "Carregar Sugestões"
- 4 cards com sugestões coloridas
- Estatísticas gerais do período
- Tabela de análise por ano
- Recomendações listadas
- Exemplos de imagens de comparação (máximo, mínimo, mediano)

- Botão para exportar como CSV

7. Seção de Upload

- Área drag-and-drop para TIFF
- Análise automática após upload
- Suporta múltiplos formatos (.tif, .tiff)

Tecnologias Frontend

- **Framework:** Bootstrap 5.3.0
- **Mapas:** Leaflet.js 1.9.4
- **Ícones:** Bootstrap Icons

- **JavaScript:** ES6 vanilla (sem jQuery)

- **Charts:** Plotly.js (via CDN)

- **Notificações:** Bootstrap Toasts

Processamento de Dados

Fluxo de Dados

...

Pasta com Imagens TIFF

?

Descoberta Automática (glob)

?

Fila de Processamento (ThreadPoolExecutor)

?

Análise por Imagem (AnalizadorLuzSatelite)

?? Validar dimensões

?? Detectar pixels nulos

?? Detectar outliers

?? Calcular estatísticas

?

Cache em JSON (processados.json)

?

Salvar em CSV (resultados_*.csv)

...

Formato de Dados

CSV Resultados

...

ano,mes,intensidade_media,intensidade_mediana,desvio_padrao,status

2014,1,32.45,31.20,12.50,ACEITA

2014,2,33.12,32.10,13.20,ACEITA

2014,3,34.89,33.95,14.10,ACEITA

...

JSON Cache

```json

{

"2014/imagem\_01.tif": {

"intensidade\_media": 32.45,

"hash\_processamento": "abc123...",

"timestamp": 1609459200,

"version": "2.0"

}

}

...

---

## Análise e Relatórios

### Relatório Padrão

Gerado para cada imagem:

...

????????????????????????????????????????????????????????????????????????????????

?      RELATÓRIO DE ANÁLISE DE INTENSIDADE DE LUZ      ?

????????????????????????????????????????????????????????????????????????????????

? RESUMO DE VALIDAÇÃO:

? Total de pixels:      250,000

? Pixels válidos:      248,750 (99.50%)

? Pixels nulos (erro):    1,200 (0.48%)

? Pixels outliers:      50 (0.02%)

? ESTATÍSTICAS DE INTENSIDADE (apenas pixels válidos):



? Média: 45.32

? Mediana: 42.15

? Mínima: 1.20

? Máxima: 248.95

? Desvio padrão: 28.45

? THRESHOLDS:

? Limite de outlier: 250.00

? Crescimento de luz: 59.53 (referência p/ comparação)

? QUALIDADE DA IMAGEM:

? Status geral: ? ACEITA

---

---

### Relatório de Tendências

Análise anual com gráficos:

...

ANÁLISE DE 10 ANOS (2014-2024)

Período: 2014-01 até 2024-12

Total de imagens: 145

Cidades analisadas: 1

ESTATÍSTICAS GERAIS:

?? Intensidade média geral: 42.15

?? Máxima anual: 47.89 (2024)

?? Mínima anual: 32.45 (2014)

?? Crescimento total: +15.44 (+47.6%)

?? Tendência: CRESCIMENTO LINEAR

#### CRESCIMENTO POR PERÍODO:

?? 2014-2018: +8.23 (+25.3%)

?? 2019-2022: +4.15 (+12.1%)

?? 2023-2024: +3.06 (+6.8%)

#### RECOMENDAÇÕES:

? Crescimento consistente indicando urbanização

? Taxa de crescimento desacelerou em 2023-2024

'''

---

## ## Automação

### ### Sistema de Agendamento

Integração com Windows Task Scheduler para processamento automático.

#### **\*\*Configuração:\*\***

- **\*\*Frequência:\*\*** Diária

- **\*\*Horário:\*\*** 02:00 (2 AM)

- **\*\*Ação:\*\*** Processa imagens novas

- **Log:** automacao.log

**Arquivo: automacao.py**

```
python
```

```
Processa novas imagens todos os dias às 2 AM
```

```
schedule.every().day.at("02:00").do(processar)
```

```
...
```

```

```

**Guias de Uso**

**1. Instalação Rápida**

```
```bash
```

1. Clonar/baixar o projeto

```
git clone <repositorio>
```

```
cd Algoritmo_de_leitura_de_luz
```

2. Criar ambiente virtual

```
python -m venv venv
```

```
venv\Scripts\activate
```

3. Instalar dependências

```
pip install -r requirements.txt
```

4. Iniciar servidor

```
python app.py
```

5. Acessar em navegador

http://localhost:5000

...

2. Primeiro Processamento

```bash

# Opção 1: Via Interface Web

# Selecionar cidade ? Clicar em "Processar Todas as Imagens"

# Opção 2: Via CLI

python main.py 1

...

**\*\*Tempo esperado:\*\*** 5-10 minutos (120 imagens)

---

### 3. Análise de Tendências

```bash

Via Interface Web

Clicar em "? Tendências" no menu

Via CLI

python main.py 2

...

****Resultado:**** Gráficos PNG + Relatório

4. Sugestão de Thresholds

...

Interface Web:

1. Selecionar cidade
2. Clicar em "? Sugestão de Thresholds"
3. Clicar em "Carregar Sugestões"

4. Análise automática dos dados

5. 4 níveis de sugestão aparecem

...

5. Comparação de Duas Imagens

...

Interface Web:

1. Selecionar primeira imagem

2. Clicar em "Comparar"

3. Selecionar segunda imagem

4. Ver resultado de crescimento/decrécimo

'''

6. Upload de Arquivo Personalizado

'''

Interface Web:

1. Ir para seção "Upload de Arquivo TIFF"
2. Arrastar arquivo ou clicar para selecionar
3. Arquivo será analisado automaticamente

'''

7. Configurar Automação

```
```bash
```

```
Via CLI
```

```
python main.py 3
```

```
Seguir instruções na tela para criar tarefa agendada
```

```
```
```

```
---
```

```
## Estrutura de Diretórios
```

'''

Algoritmo_de_leitura_de_luz/

??? app.py # Aplicação Flask

??? main.py # Menu principal

??? leitura_de_luz.py # Análise de imagens

??? processador_paralelo.py # Processamento paralelo

??? tendencias.py # Análise de tendências

??? sugestores_threshold.py # Sugestão de thresholds

??? comparador_crescimento.py # Comparação

??? automacao.py # Agendamento automático

?

??? templates/

? ??? index.html # Interface principal

?

??? static/

? ??? script.js # Lógica frontend

? ??? style.css # Estilos

? ??? assets/ # Imagens/ícones

?

??? resultados_*.csv # Dados processados

??? processados_*.json # Cache

??? coordenadas_*.json # Coordenadas

?

??? docs/

??? GUIA_RAPIDO.md

??? INSTALACAO_RAPIDA.md

??? INSTRUcoes_TESTE.md

??? DOCUMENTACAO_COMPARACAO_IMAGENS.md

'''

Requisitos Técnicos

Dependências Python

'''

Flask>=2.0.0

numpy>=1.21.0

rasterio>=1.2.0

matplotlib>=3.4.0

Pillow>=8.0.0

schedule>=1.1.0

...

Hardware Recomendado

- **CPU:** 4 cores (mínimo), 8 cores (recomendado)
- **RAM:** 8GB (mínimo), 16GB (recomendado)
- **Disco:** 10GB livres (dados + cache)
- **GPU:** Não necessária

Sistemas Operacionais

- ? Windows 10/11
- ? Linux (Ubuntu 18.04+)

- ? macOS 10.15+

Troubleshooting

Problema: "Nenhuma pasta encontrada"

****Solução:**** Verificar se as imagens estão em `C:\Users\{usuario}\Documents\`

Problema: "Erro ao analisar TIFF"

****Solução:**** Verificar se arquivo é GeoTIFF válido com `gdalinfo`

Problema: "Processamento muito lento"

****Solução:**** Verificar limite de threads, aumentar de 4 para 8

Problema: "Porta 5000 já em uso"

****Solução:**** Executar em porta diferente: ``python app.py --port 5001``

Histórico de Versões

v2.0 (Maio 2026) - ATUAL

- ? Sugestão automática de thresholds
- ? Sistema de 4 níveis de sugestão
- ? Exemplos de imagens variadas
- ? Interface web melhorada
- ? Documentação completa

v1.5 (Março 2026)

- Comparação de crescimento
- Análise de tendências
- Processamento paralelo

v1.0 (Janeiro 2026)

- Análise básica de imagens
- Interface web inicial
- API REST

Contato e Suporte

Para dúvidas ou sugestões, consulte a documentação:

- GUIA_RAPIDO.md

- INSTALACAO_RAPIDA.md

- INSTRUcoes_TESTE.md

****Desenvolvido com ?? para análise de crescimento urbano****